Digital Electronics (EC2L004)

Lecture 21: Crash Course on Verilog - Part III

Introduction

- Designers need to be able to evaluate the tradeoffs of various architectures and algorithms before they decide on the optimum architecture and algorithm to implement in hardware.

- Verilog provides designers the ability to describe design functionality in an algorithmic manner.

- Behavioral modeling represents the circuit at a very high level of abstraction.

- Design at this level resembles C programming more than it resembles digital circuit design.

Learning Objectives

- Understanding Behavioral Modeling
- Structured procedures “always” and “initial”.
- Define blocking and non-blocking procedural assignments.
- Describe event-based timing control mechanism in behavioral modeling.

Learning Objectives

- Conditional Statements/Multiway Branching
  - if-else
  - case
- Miscellaneous
  - Synchronous/Asynchronous resets
- Behavioural modelling of sequential logic storage device
- Behavioural modelling of Finite State Machines (FSMs)

Learning Objectives

- Behavioural modelling of counters
- Register Transfer Level (RTL) model of a synchronous digital system.
  - Register
  - Shift Register

Procedural Blocks

- All procedural signal assignments must be enclosed within a procedural block.
- Verilog has two types of procedural blocks: initial and always
- Each always and initial statement represents a separate activity flow in Verilog.
- Each activity flow starts at simulation time 0.
- The statements always and initial cannot be nested.

initial

- An initial block starts at time 0, executes exactly once during a simulation, and then does not execute again.
- If there are multiple initial blocks, each block starts to execute concurrently at time 0.
- Each block finishes execution independently of other blocks.
- Multiple behavioral statements must be grouped, typically using the keywords begin and end.

initial

- An initial block is not used to model synthesizable behavior.
- It is instead used within testbenches to either set the initial values of repetitive signals.
- Typically used for initialization, monitoring, waveforms and to execute one-time executable processes.

Example: initial

```
module stimulus;
reg x,y, a,b, m;

initial
    m = 1'b0;
    //single statement; does not need to be grouped
initial
    begin
        #5 a = 1'b1;
        //multiple statements; need to be grouped
        #25 b = 1'b0;
    end
initial
    begin
        #10 x = 1'b0;
        #25 y = 1'b1;
    end
initial
    #50 $finish;
endmodule
```

always

- Model a block of activity that is repeated continuously in a digital circuit
- An always block will execute forever, or for the duration of the simulation. Equivalent to an infinite loop
- Stopped only by $finish (power-off) or $stop (interrupt)
- An always block can be used to model synthesizable circuits in addition to non-synthesizable behavior in test benches.

Example: always

```
module clock_gen (output reg clock);
//Initialize clock at time zero
initial
    clock = 1'b0;
//Toggle clock every half-cycle (time period = 20)
always
    #10 clock = ~clock;
initial
    #1000 $finish;
endmodule
```

Procedural Assignment

- Verilog uses procedural assignment to model signal assignments that are based on an event.
- An event is most commonly a transition of a signal.
- This provides the ability to model sequential logic circuits such as D-flip-flops and finite state machines by triggering assignments of a clock edge.
- A procedural assignment can also be used to model combinational logic circuits by making signal assignments when any of the inputs to the model change.

Procedural Assignment

- The value placed on a variable will remain unchanged until another procedural assignment updates the variable with a different value.

- Procedural assignment also supports standard programming constructs such as if-else decisions, case statements, and loops.

- These are unlike continuous assignments, where one assignment statement can cause the value of the right-hand-side expression to be continuously placed onto the left-hand-side net.

Procedural Assignments Cont'd

- The left-hand side of a procedural assignment <lvalue> can be one of the following:
  - A reg, integer, real, or time register variable or a memory element
  - A bit select of these variables (e.g., addr[0])
  - A part select of these variables (e.g., addr[31:16])
  - A concatenation of any of the above

- The right-hand side can be any expression that evaluates to a value.

- There are two types of procedural assignment statements: blocking and nonblocking.

Blocking Assignments

- A blocking assignment is denoted with the = symbol and the evaluation and assignment of each statement takes place immediately.

- When this behavior is coupled with a sensitivity list that contains all the inputs to the system, this approach can model synthesizable combinational logic circuits.

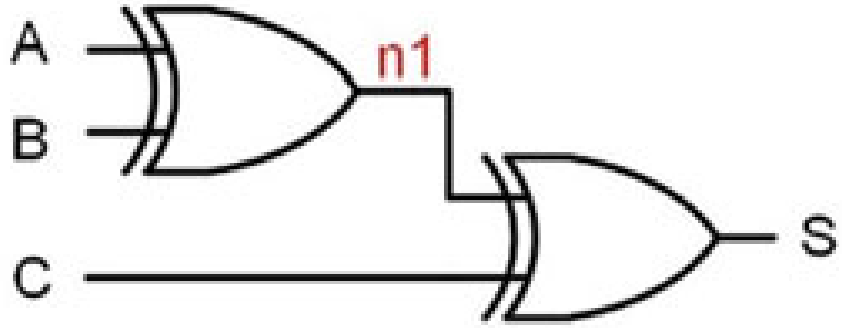
Blocking Assignments

Why should we use blocking assignments instead of continuous assignments to model combinational logic?

- Blocking statements have more advanced programming constructs are supported within Verilog procedural blocks.

Blocking Assignments

```
module BlcokingExample ( output reg S,
                         input wire A, B, C);

    reg n1;

    always @(A, B, C)
        begin
            n1 = A ^ B; // statement 1
            S = n1 ^ C; // statement 2
        end
endmodule
```



Blocking Assignments Cont'd

- Assignments can block execution in another concurrent construct also

```
always @(posedgeclk)
begin
    word[15:8] = word[ 7:0];
    word[ 7:0] = word[15:8];
end
//swap bytes in word???
```

```
always @(posedgeclk)
begin
    word[ 7:0] = word[15:8];
    word[15:8] = word[ 7:0];
end
//swap bytes in word???
```

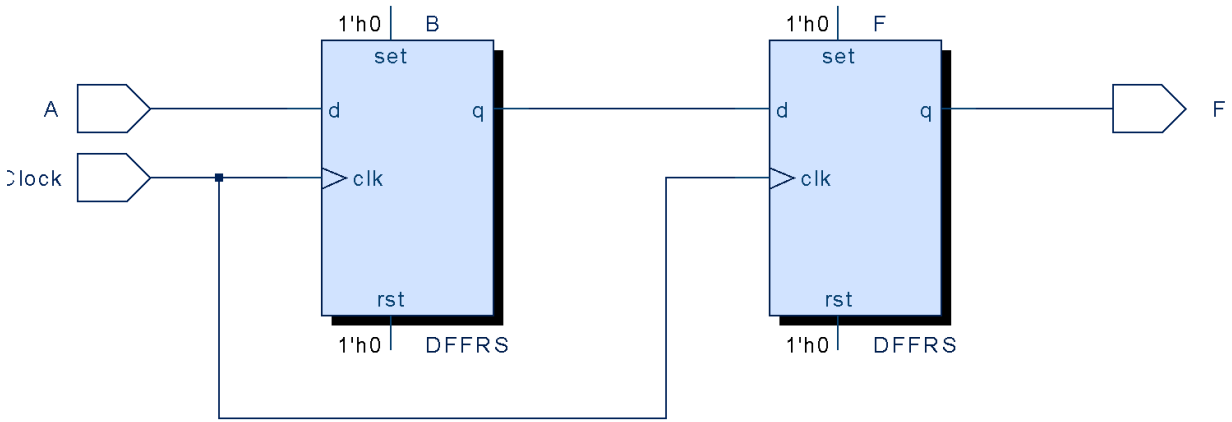
Non-blocking Assignments

- Evaluated and assigned in two steps:
  - The right-hand side is evaluated immediately
  - The assignment to the left-hand side is postponed until other evaluations in the current time step are completed
  - Execution flow within the procedure continues until a timing control is encountered (flow is not blocked)
  - Operator is <=

```
always @(posedgeclk)
begin
    word[15:8] <= word[ 7:0];
    word[ 7:0] <= word[15:8];
end
//can swap the order of statements
```

Non-blocking Assignments

```
module NonBlocking ( output reg F,
                     input wire A,
                     input wire Clock);

        reg B;

        always@(posedge Clock)
        begin
                B <= A;  //statement 1
                F <= B;  //statement 2
        end
endmodule
```



Non-blocking Assignments Cont'd

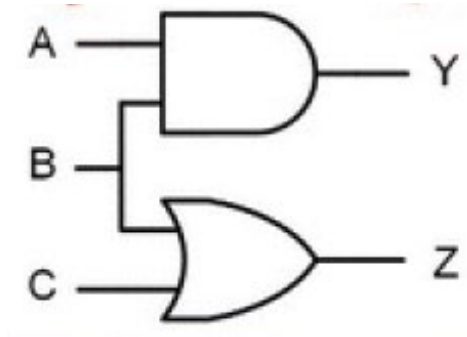
```
//Illustration 1: Two concurrent always blocks with blocking
//statements
always @(posedge clock)
a = b;
always @(posedge clock)
b = a;
//Illustration 2: Two concurrent always blocks with nonblocking
//statements
always @(posedge clock)
a <= b;
always @(posedge clock)
b <= a;
```

Blocking Vs Non-Blocking

- The difference between blocking and non-blocking assignments is subtle and is often one of the most difficult concepts to grasp.

```
module blocking ( output reg Y, Z,
                  input wire A, B, C);

        always@(A, B, C)
        begin
                Y = A & B;
                Z = B | C;
        end
endmodule
```

```
module nonblocking (output reg Y, Z,
                    input wire A, B, C);

        always@(A, B, C)
        begin
                Y <= A & B;
                Z <= B | C;
        end
endmodule
```



Blocking Vs Non-Blocking

- Both blocking and non-blocking assignments *can* produce the same results when they either contains a single assignment or a list of assignments that don't have any signal interdependencies.

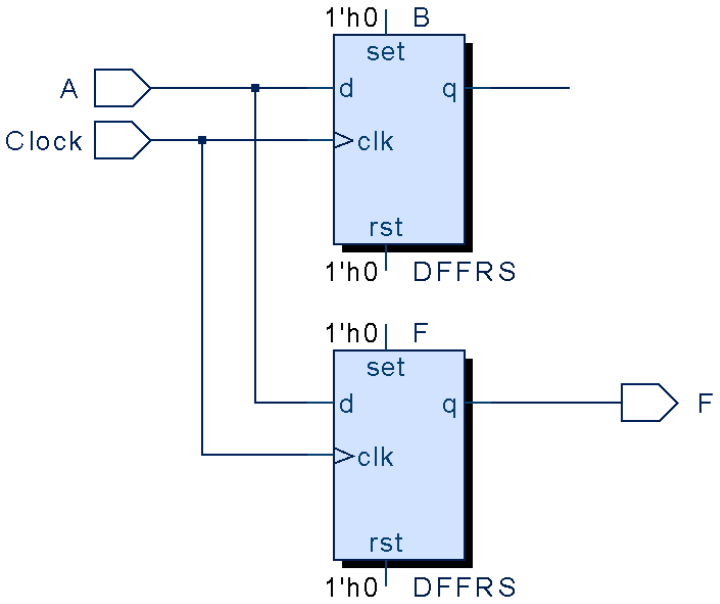
- A *signal interdependency* refers to when a signal that is the target of an assignment (i.e., on the LHS of an assignment) is used as an argument (i.e., on the RHS of an assignment) in subsequent statements.

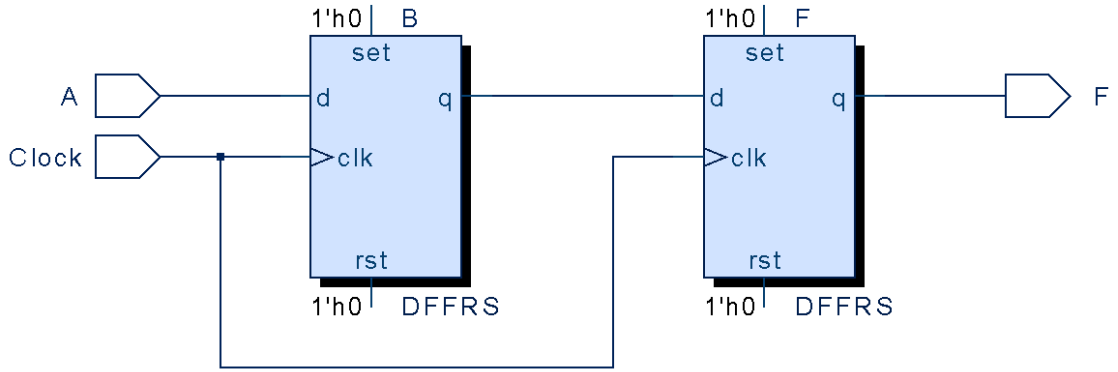
Blocking Vs Non-Blocking

```
module blcoking (output reg F,
                 input wire A,
                 input wire Clock);

reg B;

always@(posedge Clock)
begin
        B = A;
        F = B;
end
endmodule
```

```
module nonblcoking (output reg F,
                    input wire A,
                    input wire Clock);

reg B;

always@(posedge Clock)
begin
        B <= A;  //statement 1
        F <= B;  //statement 2
end
endmodule
```

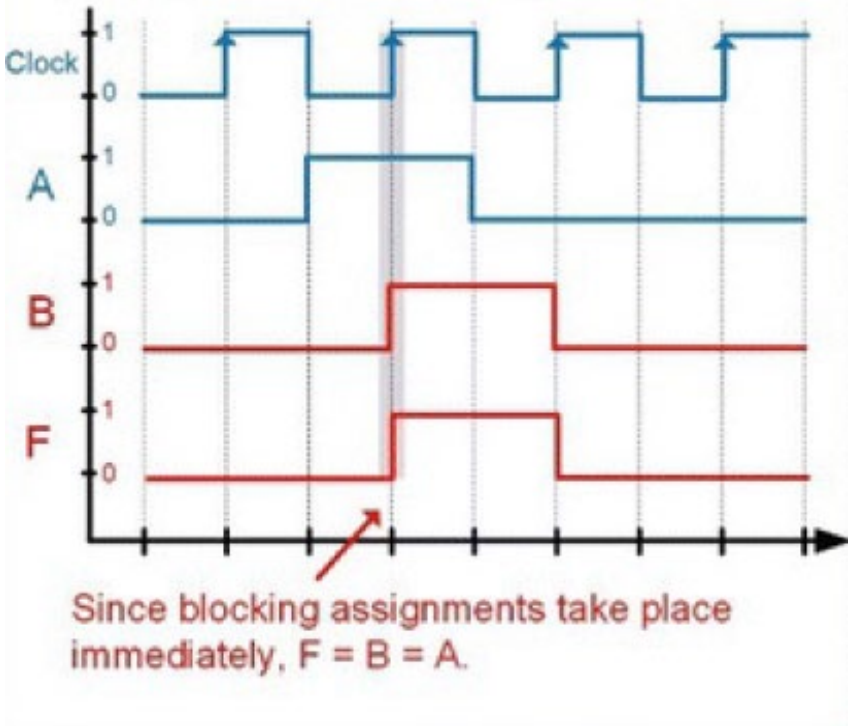
Blocking Vs Non-Blocking



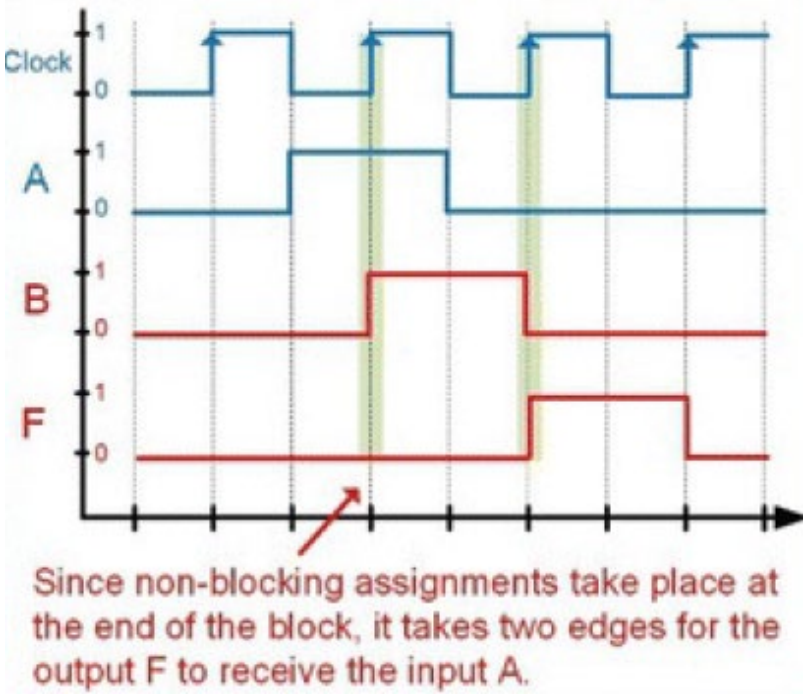


Blocking Vs Non-Blocking

Timing Waveforms: Blocking Assignment



Non-Blocking Assignment



Blocking Vs Non-Blocking

- There are two design guidelines that can make creating accurate, synthesizable models straightforward.

- They are:

  - When modeling **combinational logic**, use blocking assignments and list every input in the sensitivity list.

  - When modeling **sequential logic**, use *non-blocking assignments* and *only list the clock and reset lines* (if applicable) in the sensitivity list.

Timing Controls

Timing Controls

- Timing controls provide a way to specify the simulation time at which procedural statements will execute.

- There are three methods of timing control:
  - Event-based timing control, and
  - Level-sensitive timing control

Event-Based Timing Control

- An event is the change in the value on a register or a net.
- Events can be utilized to trigger execution of a statement or a block of statements.
- There are four types of event-based timing control:
  - regular event control,
  - named event control,
  - event OR control, and
  - level sensitive timing control.

Regular Event Control

- The @ symbol is used to specify an event control.
- Statements can be executed on changes in signal value or at a positive or negative transition of the signal value.

```
@(clock) q = d; //q = d is executed whenever signal clock changes value
@(posedge clock) q = d; //q = d is executed whenever signal clock does
//a positive transition ( 0 to 1,x or z,
// x to 1, z to 1 )
@(negedge clock) q = d; //q = d is executed whenever signal clock does
//a negative transition ( 1 to 0,x or z,
//x to 0, z to 0)
q = @(posedge clock) d; //d is evaluated immediately and assigned
//to q at the positive edge of clock
```

Event OR control

- Sometimes a transition on any one of multiple signals or events can trigger the execution of a statement or a block of statements. This is expressed as an OR of events or signals.

```
//A level-sensitive latch with asynchronous reset
always @( reset or clock or d)
//Wait for reset or clock or d to
change
begin
if (reset) //if reset signal is high, set q to 0.
q = 1'b0;
else if(clock) //if clock is high, latch input
q = d;
end
```

Example: Sensitivity List & Comma Operator

```
//A level-sensitive latch with asynchronous reset
always @( reset, clock, d)
//Wait for reset or clock or d to
change
begin
if (reset) //if reset signal is high, set q to 0.
q = 1'b0;
else if(clock) //if clock is high, latch input
q = d;
end
//A positive edge triggered D flipflop with asynchronous falling
//reset can be modeled as shown below
always @(posedge clk, negedge reset) //Note use of comma operator
if(!reset)
q <=0;
else
q <=d;
```

Example: Use of @* Operator

```
//Combination logic block using the or operator
//Cumbersome to write and it is easy to miss one input to the block
always @(a or b or c or d or e or f or g or h or p or m)
begin
out1 = a ? b+c : d+e;
out2 = f ? g+h : p+m;
end
//Instead of the above method, use @(*) symbol
//Alternately, the @* symbol can be used
//All input variables are automatically included in the
//sensitivity list.
always @(*)
begin
out1 = a ? b+c : d+e;
out2 = f ? g+h : p+m;
end
```

Sensitivity List

Conditional Programming Constructs

Conditional Programming Constructs

- Procedural blocks provide the conditional programming constructs such as if-else decisions, case statements, and loops.

- These constructs are only available within a procedural block.

- They can be used to model both combinational and sequential logic.

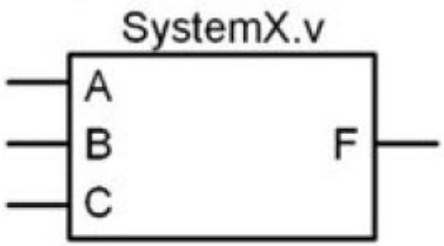
if-else Statements

- An if-else statement provides a way to make conditional signal assignments based on Boolean conditions.
- The if portion of statement is followed by a Boolean condition that if evaluated TRUE will cause the signal assignment listed after it to be performed.
- If the Boolean condition is evaluated FALSE, the statements listed after the else portion are executed.
- If multiple statements are to be executed in either the if or else portion, then the statement group keywords begin/end need to be used.

if-else Statements

```
//Type 1 conditional statement. No else statement.
//Statement executes or does not execute.
if (<expression>) true_statement ;
//Type 2 conditional statement. One else statement
//Either true_statement or false_statement is evaluated
if (<expression>) true_statement ; else false_statement ;
//Type 3 conditional statement. Nested if-else-if.
//Choice of multiple statements. Only one is executed.
if (<expression1>) true_statement1 ;
else if (<expression2>) true_statement2 ;
else if (<expression3>) true_statement3 ;
else default_statement ;
```

if-else Statements



A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

```
module SystemX
   (output reg F,
    input  wire A, B, C);

   always @ (A, B, C)
     begin
       if      (A==1'b0 && B==1'b0 && C==1'b0)
         F = 1'b1;
       else if (A==1'b0 && B==1'b1 && C==1'b0)
         F = 1'b1;
       else if (A==1'b1 && B==1'b1 && C==1'b0)
         F = 1'b1;
       else
         F = 1'b0;
     end

endmodule
```

Multiway Branching

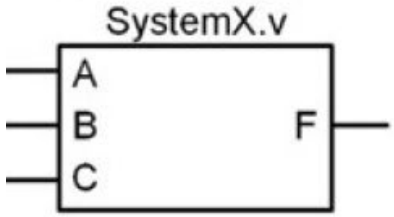
- The case statement
  - This compares 0,1,x and z of the condition, bit by bit with the different case options.
  - If width of condition and a case option are unequal, they are zero filled to match the bid width of the widest of both.

```
case (<input_name>)
    input_val_1 : statement_1
    input_val_2 : statement_2
                      :
    input_val_n : statement_n
    default     : default_statement
endcase
```

Multiway Branching

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
reg out;
always @(s1 or s0 or i0 or i1 or i2 or i3)
case ({s1, s0})
//Switch based on concatenation of control signals
2'd0 : out = i0;
2'd1 : out = i1;
2'd2 : out = i2;
2'd3 : out = i3;
default: $display("Invalid control signals");
endcase
endmodule
```

Case Statement



A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

```
module SystemX
  (output reg F,
   input  wire A, B, C);

  always @ (A, B, C)
    begin
      case ( {A,B,C} )
         3'b000  : F = 1'b1;
         3'b001  : F = 1'b0;
         3'b010  : F = 1'b1;
         3'b011  : F = 1'b0;
         3'b100  : F = 1'b0;
         3'b101  : F = 1'b0;
         3'b110  : F = 1'b1;
         3'b111  : F = 1'b0;
         default : F = 1'bX;
      endcase
    end

endmodule
```

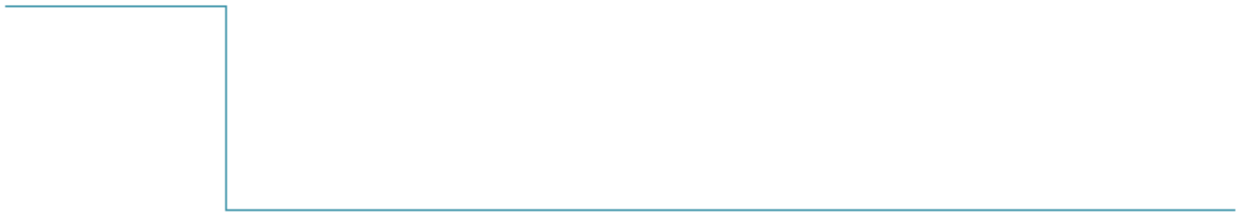
```
case ( {A,B,C} )
   3'b000  : F = 1'b1;
   3'b010  : F = 1'b1;
   3'b110  : F = 1'b1;
   default : F = 1'b0;
endcase
```

```
case ( {A,B,C} )
    3'b000, 3'b010, 3'b111  : F = 1'b1;
    default                 : F = 1'b0;
endcase
```

Synchronous Reset / Asynchronous Reset

- Why is reset used ?

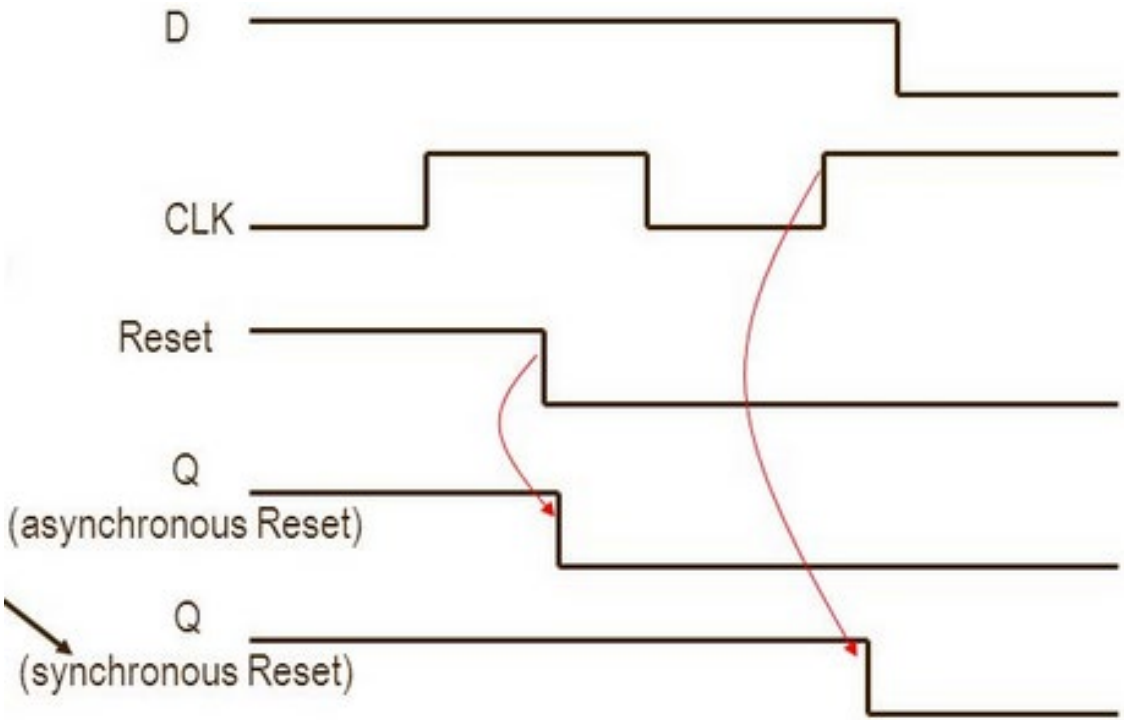
  - Reset is used to initialize the hardware design of a system and to make system to known state from unknown state.



Exemplary Reset Signal

Synchronous Reset / Asynchronous Reset

- Synchronous reset : A synchronous reset will reset the circuit at active edge of clock.
- Asynchronous reset : Asynchronous reset will reset the circuit asynchronously i.e. no matter with the clock.



Synchronous Reset – D Flip-Flop

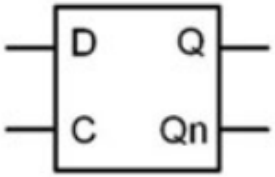
```
module dff_sync_reset (
data    , // Data Input
clk     , // Clock Input
reset   , // Reset input
q         // Q output
);
//-----------Input Ports---------------
input data, clk, reset ;

//-----------Output Ports---------------
output q;

//------------Internal Variables--------
reg q;

//-------------Code Starts Here---------
always @ ( posedge clk)
if (~reset) begin
  q <= 1'b0;
end  else begin
  q <= data;
end

endmodule //End Of Module dff sync reset
```

Asynchronous Reset – D Flip-Flop

```
module dff_async_reset (
data   , // Data Input
clk     , // Clock Input
reset , // Reset input
q           // Q output
);
//-----------Input Ports---------------
input data, clk, reset ;

//-----------Output Ports---------------
output q;

//------------Internal Variables--------
reg q;

//-------------Code Starts Here---------
always @ ( posedge clk or negedge reset)
if (~reset) begin
  q <= 1'b0;
end  else begin
  q <= data;
end

endmodule //End Of Module dff async reset
```

Behavioral Modeling of Sequential Logic

D-Latch

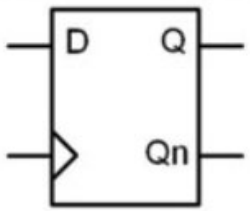


C	D	Q	Qn
0	X	Last Q	Last Qn
1	0	0	1
1	1	1	0

```
module dlatch (output reg  Q, Qn,
               input  wire C, D);

  always @ (C or D)
    if (C == 1'b1)
      begin
        Q  <= D;
        Qn <= ~D;
      end

endmodule
```

Behavioral model of a D-latch in Verilog

D-Flip-Flop

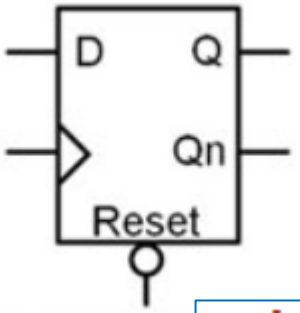


Clk	D	Q	Qn
0	X	Last Q	Last Qn
1	X	Last Q	Last Qn
↑	0	0	1
↑	1	1	0

```
module dflipflop (output reg  Q, Qn,
                  input  wire Clock, D);

  always @ (posedge Clock)
     begin
        Q  <= D;
        Qn <= ~D;
     end

endmodule
```

Behavioral model of a D-flip-flop in Verilog

D-Flip-Flop with Asynchronous Reset

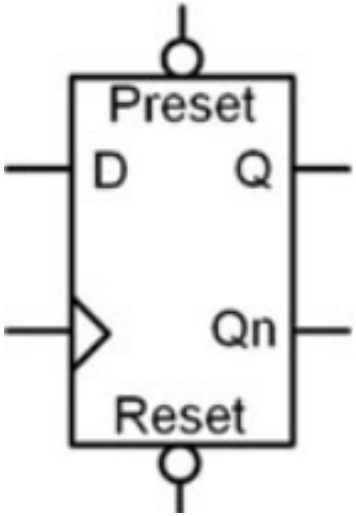


$\overline{R}$	Clk	D	Q	Qn
0	X	X	0	1
1	0	X	Last Q	Last Qn
1	1	X	Last Q	Last Qn
1	↑	0	0	1
1	↑	1	1	0

```
module dflipflop (output reg  Q, Qn,
                  input  wire Clock, Reset, D);

  always @ (posedge Clock or negedge Reset)
    if (!Reset)
      begin
        Q  <= 1'b0;
        Qn <= 1'b1;
      end
    else
       begin
         Q  <= D;
         Qn <= ~D;
       end

endmodule
```

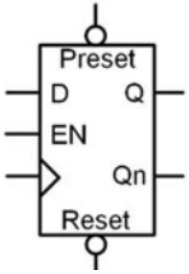
D-FF with Asynchronous Reset and Preset

```
module dflipflop (output reg  Q, Qn,
                  input  wire Clock, Reset, Preset, D);

  always @ (posedge Clock or negedge Reset or negedge Preset)
    if (!Reset)
      begin
        Q  <= 1'b0;
        Qn <= 1'b1;
      end
    else if (!Preset)
      begin
        Q  <= 1'b1;
        Qn <= 1'b0;
      end
    else
       begin
         Q  <= D;
         Qn <= ~D;
       end

endmodule
```



R	P	Clk	D	Q	Qn
0	X	X	X	0	1
1	0	X	X	1	0
1	1	0	X	Last Q	Last Qn
1	1	1	X	Last Q	Last Qn
1	1	↓	0	0	1
1	1	↓	1	1	0

D-FF with synchronous enable in Verilog




```
module dflipflop (output reg  Q, Qn,
                  input  wire Clock, Reset, Preset, D, EN);

  always @ (posedge Clock or negedge Reset or negedge Preset)
    if (!Reset)
      begin
        Q  <= 1'b0;
        Qn <= 1'b1;
      end
    else if (!Preset)
      begin
        Q  <= 1'b1;
        Qn <= 1'b0;
      end
    else if (EN)
       begin
         Q  <= D;
         Qn <= ~D;
       end

endmodule
```

Since EN is not listed in the sensitivity list it does not trigger the block when it transitions. This "if" statement is only reached if there is a rising edge of the clock. This models an enable that is synchronous to the clock.

Modeling Finite State Machines in Verilog

FSMs in Verilog

- Finite state machines can be easily modeled using the behavioral constructs.
- The most common modeling practice for FSMs is to declare two signals of type reg that are called current_state and next_state.
- Then a parameter is declared for each descriptive state name in the state diagram.
- A parameter also requires a value, so the state encoding can be accomplished during the parameter declaration.

FSMs in Verilog

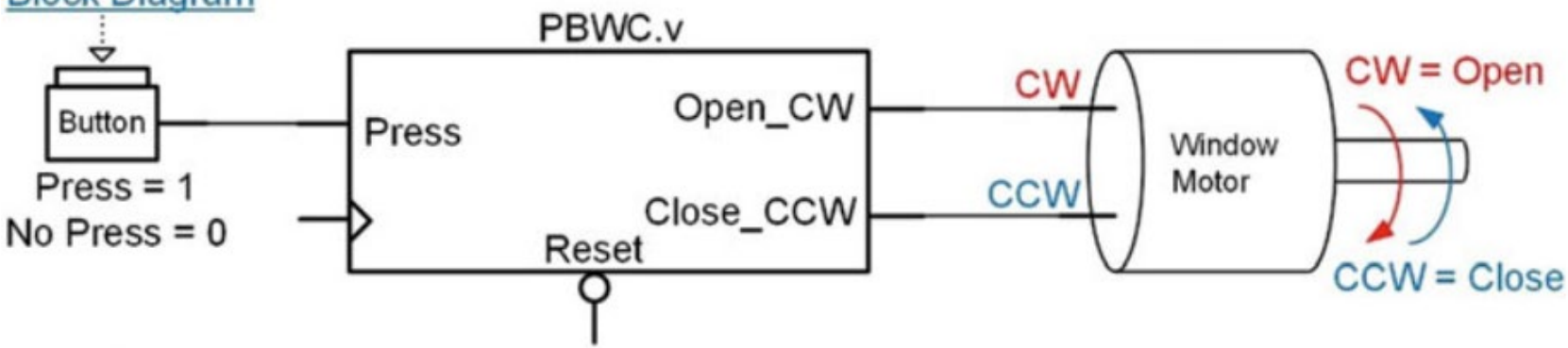
- Once the signals and parameters are created, all of the procedural assignments in the state machine model can use the descriptive state names in their signal assignments.

- Within the Verilog state machine model, three separate procedural blocks are used to describe each of the functional blocks, state memory, next state logic, and output logic

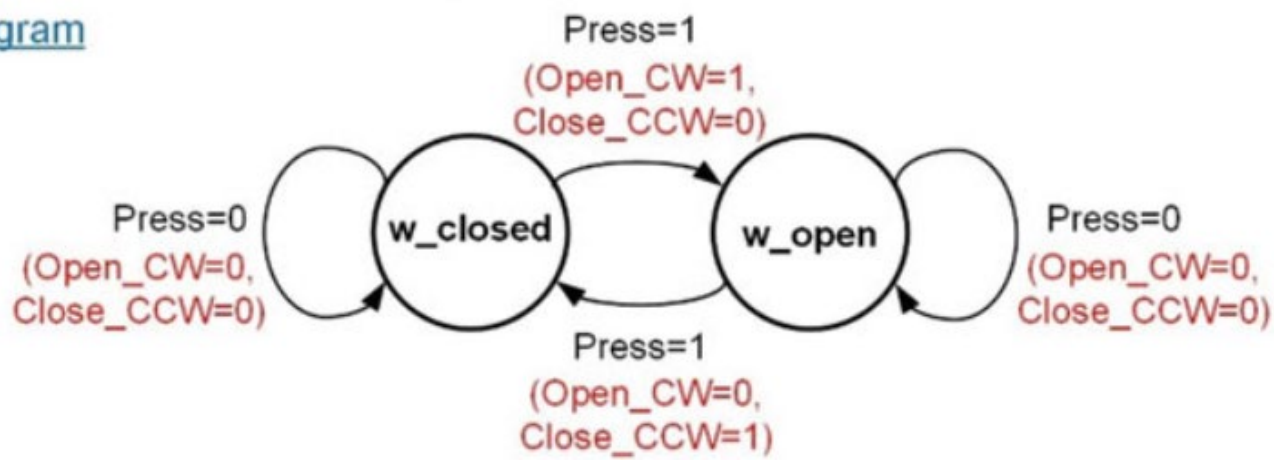
Example: Push-Button Window Controller

The window controller will send the appropriate control signals to a motor to open or close it whenever a button is pressed. The system must keep track whether the window is open or closed in order to send the correct signal, thus a state machine is needed. The block diagram and state diagram for this system is shown below.

Block Diagram

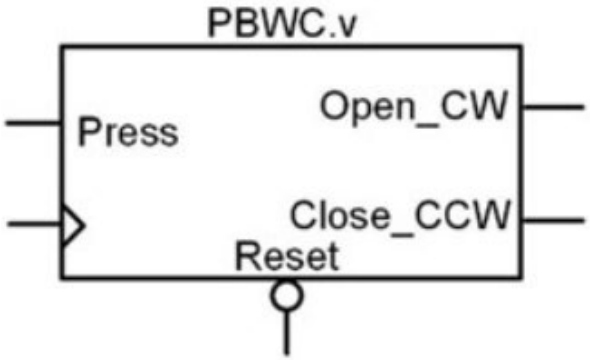


State Diagram



Example: Push-Button Window Controller

Example: Push-Button Window Controller in Verilog – Port Definition




```
module PBWC (output reg  Open_CW, Close_CCW,
             input  wire Clock, Reset, Press);

                    :
                    :
```

Outputs are defined as type reg while inputs are defined of type wire.

Example: Push-Button Window Controller

Modeling States

```
reg       current_state, next_state;
parameter w_closed = 1'b0,
          w_open   = 1'b1;
```

The State Memory Block

```
always @ (posedge Clock or negedge Reset)
  begin: STATE_MEMORY
    if (!Reset)
      current_state <= w_closed;
    else
      current_state <= next_state;
  end
```

Example: Push-Button Window Controller

The Next State Logic Block

```
always @ (current_state or Press)
  begin: NEXT_STATE_LOGIC
    case (current_state)
      w_closed : if (Press == 1'b1)
                     next_state = w_open;
                 else
                     next_state = w_closed;
      w_open   : if (Press == 1'b1)
                     next_state = w_closed;
                 else
                     next_state = w_open;
      default  : next_state = w_closed;
    endcase
  end
```

Example: Push-Button Window Controller

The Output Logic Block

```
always @ (current_state or Press)
  begin: OUTPUT_LOGIC
    case (current_state)
      w_closed : if (Press == 1'b1)
                   begin
                     Open_CW   = 1'b1;
                     Close_CCW = 1'b0;
                   end
                 else
                   begin
                     Open_CW   = 1'b0;
                     Close_CCW = 1'b0;
                   end

      w_open   : if (Press == 1'b1)
                   begin
                     Open_CW   = 1'b0;
                     Close_CCW = 1'b1;
                   end
                 else
                   begin
                     Open_CW   = 1'b0;
                     Close_CCW = 1'b0;
                   end

       default : begin
                     Open_CW   = 1'b0;
                     Close_CCW = 1'b0;
                 end
    endcase

  end
```

Example: Push-Button Window Controller

```
module PBWC (output reg  Open_CW, Close_CCW,
             input  wire Clock, Reset, Press);

  reg       current_state, next_state;
  parameter w_closed = 1'b0,
            w_open   = 1'b1;
//-----------------------------------------------
// STATE MEMORY

  always @ (posedge Clock or negedge Reset)
    begin: STATE_MEMORY
      if (!Reset)
        current_state <= w_closed;
      else
        current_state <= next_state;
    end
//-----------------------------------------------
// NEXT STATE LOGIC

  always @ (current_state or Press)
    begin: NEXT_STATE_LOGIC
      case (current_state)
        w_closed : if (Press == 1'b1)
                     next_state = w_open;
                   else
                     next_state = w_closed;
        w_open   : if (Press == 1'b1)
                     next_state = w_closed;
                   else
                     next_state = w_open;
        default  : next_state = w_closed;
      endcase
    end
//-----------------------------------------------
// OUTPUT LOGIC

  always @ (current_state or Press)
    begin: OUTPUT_LOGIC
      case (current_state)
        w_closed : if (Press == 1'b1)
                     begin
                       Open_CW  = 1'b1;
                       Close_CCW = 1'b0;
                     end
                   else
                     begin
                       Open_CW  = 1'b0;
                       Close_CCW = 1'b0;
                     end
        w_open   : if (Press == 1'b1)
                     begin
                       Open_CW  = 1'b0;
                       Close_CCW = 1'b1;
                     end
                   else
                     begin
                       Open_CW  = 1'b0;
                       Close_CCW = 1'b0;
                     end

        default : begin
                       Open_CW  = 1'b0;
                       Close_CCW = 1'b0;
                  end
      endcase
    end
endmodule
```

Declaration of state variables and state encoding.

State memory block. This is sequential logic so non-blocking assignments are used. This block only makes assignments to the signal "current_state".

Next state logic block. This is combinational logic so blocking assignments are used. This block only makes assignments to the signal "next_state".

Output logic block. This is combinational logic so blocking assignments are used. Since this is a Mealy machine, the current state and input are listed in the sensitivity list. This block only makes assignments to the outputs "Open_CW" and "Close_CCW".

Binary Counter- Single Procedural Block

```
module Counter_4bit_Up (output reg [3:0] CNT,
                        input  wire Clock, Reset);

  always @ (posedge Clock or negedge Reset)
    begin: COUNTER
      if (!Reset)
        CNT <= 0;
      else
        CNT <= CNT + 1;
    end

endmodule
```

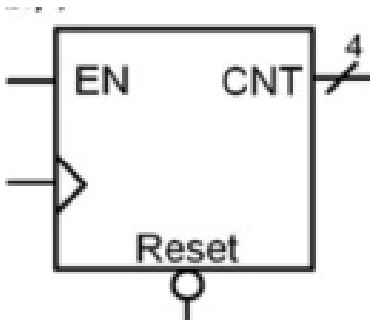
Counters with Range Checking

```
module Counter_4bit_Up (output reg [3:0] CNT,
                        input  wire Clock, Reset);

  always @ (posedge Clock or negedge Reset)
    begin: COUNTER
      if (!Reset)
        CNT <= 0;
      else
        if (CNT == 10)
          CNT <= 0;
        else
          CNT <= CNT + 1;
    end

endmodule
```

A nested if-else statement checks if the counter has reached its maximum value. If it has, it is reset back to zero. If it hasn't, it increments.

Binary Counter with Enable in Verilog

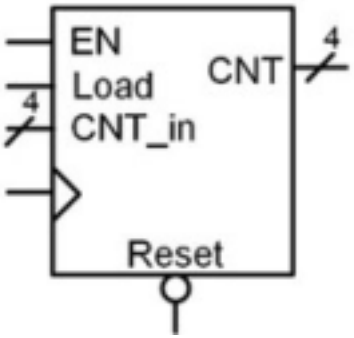
```
module Counter_4bit_Up (output reg [3:0] CNT,
                        input  wire Clock, Reset, EN);

  always @ (posedge Clock or negedge Reset)
    begin: COUNTER
      if (!Reset)
        CNT <= 0;
      else
        if (EN)
          CNT <= CNT + 1;
    end

endmodule
```



The EN is synchronous to the clock, so its logic is nested beneath the portion of the main if-else clause that handles the behavior when the counter receives a rising edge of clock.

Counters with Loads

```
module Counter_4bit_Up (output reg [3:0] CNT,
                        input  wire Clock, Reset, EN, Load,
                        input  wire [3:0] CNT_in);

  always @ (posedge Clock or negedge Reset)
    begin: COUNTER
      if (!Reset)
        CNT <= 0;
      else
        if (EN)
           if (Load)
              CNT <= CNT_in;
           else
              CNT <= CNT + 1;
    end

endmodule
```



A nested if-else statement is used to load CNT with CNT_in when the Load signal is asserted and the counter receives a rising edge of clock.

References

- Chapter 7, Verilog HDL by Samir Palnitkar, Second Edition.

- Chapter 9, Introduction to Logic Circuits & Logic Design with Verilog by Brock J. LaMeres

- Disclaimer: “I don’t claim the ownership of all the slides, some of the material is picked up from various publicly available sources on the internet”.

Thank you